6_3

September 20, 2026

1 Bài 6.3 — Package FIM (PyFIM) của Christian Borgelt

Yêu cầu: - Cài đặt package FIM từ nguồn http://www.borgelt.net - Chạy thử giải thuật Apriori trên dữ liệu của bài trước (Market-Basket, slide 3) - Tham khảo source code `pyfim/ex` để biết cách sử dụng

Source: http://www.borgelt.net/src/pyfim.zip

[1]:
```
import fim
import pandas as pd

print("fim module:", fim.__file__)
print("version   :", fim.__version__ if hasattr(fim, "__version__") else "6.28")
print("các hàm   :", [f for f in dir(fim) if not f.startswith("_")])
```

```
fim module: /mnt/datassd/CodeHub/Science/WILD/DataMining/6/fim.so
version   : 6.28
các hàm   : ['accretion', 'apriacc', 'apriori', 'arules', 'carpenter', 'eclat',
'estpsp', 'fim', 'fpgrowth', 'genpsp', 'ista', 'patred', 'psp2bdr', 'relim',
'sam']
```

1.1 2. Cú pháp hàm `fim.apriori` (tham khảo `pyfim/ex/testfim.py`)

```
apriori(tracts, target='s', supp=10, conf=80, zmin=1, zmax=None,
        report='a', eval='x', thresh=10, algo='b', mode='')
```

Tham số	Ý nghĩa
`tracts`	danh sách giao dịch, mỗi giao dịch là danh sách item
`target`	`s` = mọi tập phổ biến, `c` = closed, `m` = maximal, `g` = generator, `r` = **luật kết hợp**
`supp`	minSup — **số dương = phần trăm**, số âm = số giao dịch tuyệt đối
`conf`	minConf tính theo **phần trăm** (chỉ dùng khi `target='r'`)
`zmin` / `zmax`	số item tối thiểu / tối đa trong một tập
`report`	chuỗi quy định giá trị trả kèm: `a` count tuyệt đối, `s` support dạng phân số, `S` support dạng %, `c` confidence phân số, `C` confidence %, `l` lift

Lưu ý `supp=40` nghĩa là **40%**, không phải 0.4.

1.2 3. Dữ liệu bài trước (Market-Basket — slide 3)

[3]:
```
transactions = {
    1: ["Bread", "Milk"],
    2: ["Bread", "Diaper", "Beer", "Eggs"],
    3: ["Milk", "Diaper", "Beer", "Coke"],
    4: ["Bread", "Milk", "Diaper", "Beer"],
    5: ["Bread", "Milk", "Diaper", "Coke"],
}

tracts = list(transactions.values())    # định dạng PyFIM yêu cầu: list of lists
N = len(tracts)

MIN_SUP_PCT  = 40    # minSup = 40%  (>= 2/5 giao dịch)
MIN_CONF_PCT = 60    # minConf = 60%

for tid, items in transactions.items():
    print(tid, items)
print(f"\nN = {N}, minSup = {MIN_SUP_PCT}% ({MIN_SUP_PCT * N // 100} giao␣
 ↪dịch), minConf = {MIN_CONF_PCT}%")
```

```
1 ['Bread', 'Milk']
2 ['Bread', 'Diaper', 'Beer', 'Eggs']
3 ['Milk', 'Diaper', 'Beer', 'Coke']
4 ['Bread', 'Milk', 'Diaper', 'Beer']
5 ['Bread', 'Milk', 'Diaper', 'Coke']

N = 5, minSup = 40% (2 giao dịch), minConf = 60%
```

1.3 4. Tập phổ biến — `target='s'`

`report='aS'` → mỗi kết quả là `(itemset, count, support%)`.

[4]:
```
itemsets = fim.apriori(tracts, target='s', supp=MIN_SUP_PCT, zmin=1,␣
 ↪report='aS')

freq_df = pd.DataFrame([
    {"k": len(s), "itemset": "{" + ", ".join(sorted(s)) + "}", "count": cnt,␣
 ↪"support": pct / 100}
    for s, cnt, pct in itemsets
]).sort_values(["k", "support", "itemset"], ascending=[True, False, True]).
 ↪reset_index(drop=True)

print("Số tập phổ biến:", len(freq_df))
freq_df
```

Số tập phổ biến: 17

[4]:

	k	itemset	count	support
0	1	{Bread}	4	0.8
1	1	{Diaper}	4	0.8
2	1	{Milk}	4	0.8
3	1	{Beer}	3	0.6
4	1	{Coke}	2	0.4
5	2	{Beer, Diaper}	3	0.6
6	2	{Bread, Diaper}	3	0.6
7	2	{Bread, Milk}	3	0.6
8	2	{Diaper, Milk}	3	0.6
9	2	{Beer, Bread}	2	0.4
10	2	{Beer, Milk}	2	0.4
11	2	{Coke, Diaper}	2	0.4
12	2	{Coke, Milk}	2	0.4
13	3	{Beer, Bread, Diaper}	2	0.4
14	3	{Beer, Diaper, Milk}	2	0.4
15	3	{Bread, Diaper, Milk}	2	0.4
16	3	{Coke, Diaper, Milk}	2	0.4

[5]:
```
for k, g in freq_df.groupby("k"):
    print(f"===== F_{k} ({len(g)} tập) =====")
    for _, r in g.iterrows():
        print(f"  {r['itemset']:<35} count = {r['count']}   support =␣
↪{r['support']:.2f}")
    print()
```

```
===== F_1 (5 tập) =====
  {Bread}                             count = 4   support = 0.80
  {Diaper}                            count = 4   support = 0.80
  {Milk}                              count = 4   support = 0.80
  {Beer}                              count = 3   support = 0.60
  {Coke}                              count = 2   support = 0.40

===== F_2 (8 tập) =====
  {Beer, Diaper}                      count = 3   support = 0.60
  {Bread, Diaper}                     count = 3   support = 0.60
  {Bread, Milk}                       count = 3   support = 0.60
  {Diaper, Milk}                      count = 3   support = 0.60
  {Beer, Bread}                       count = 2   support = 0.40
  {Beer, Milk}                        count = 2   support = 0.40
  {Coke, Diaper}                      count = 2   support = 0.40
  {Coke, Milk}                        count = 2   support = 0.40

===== F_3 (4 tập) =====
  {Beer, Bread, Diaper}               count = 2   support = 0.40
  {Beer, Diaper, Milk}                count = 2   support = 0.40
```

```
  {Bread, Diaper, Milk}                count = 2   support = 0.40
  {Coke, Diaper, Milk}                 count = 2   support = 0.40
```

1.3.1 Tập phổ biến đóng và tối đại

PyFIM hỗ trợ sẵn, chỉ cần đổi `target`: - `target='c'` — **closed**: không có tập cha nào có cùng support - `target='m'` — **maximal**: không có tập cha nào còn phổ biến

[6]:
```
for target, ten in [("c", "CLOSED"), ("m", "MAXIMAL")]:
    res = fim.apriori(tracts, target=target, supp=MIN_SUP_PCT, report='aS')
    print(f"--- {ten} frequent itemsets ({len(res)}) ---")
    for s, cnt, pct in sorted(res, key=lambda x: (-len(x[0]), sorted(x[0]))):
        print(f"  {{{', '.join(sorted(s))}}}".ljust(40) + f"count = {cnt}  ␣
↪support = {pct/100:.2f}")
    print()
```

```
--- CLOSED frequent itemsets (11) ---
  {Beer, Bread, Diaper}                count = 2  support = 0.40
  {Beer, Diaper, Milk}                 count = 2  support = 0.40
  {Bread, Diaper, Milk}                count = 2  support = 0.40
  {Coke, Diaper, Milk}                 count = 2  support = 0.40
  {Beer, Diaper}                       count = 3  support = 0.60
  {Bread, Diaper}                      count = 3  support = 0.60
  {Bread, Milk}                        count = 3  support = 0.60
  {Diaper, Milk}                       count = 3  support = 0.60
  {Bread}                              count = 4  support = 0.80
  {Diaper}                             count = 4  support = 0.80
  {Milk}                               count = 4  support = 0.80

--- MAXIMAL frequent itemsets (4) ---
  {Beer, Bread, Diaper}                count = 2  support = 0.40
  {Beer, Diaper, Milk}                 count = 2  support = 0.40
  {Bread, Diaper, Milk}                count = 2  support = 0.40
  {Coke, Diaper, Milk}                 count = 2  support = 0.40
```

1.4 5. Luật kết hợp — `target='r'`

`report='aSCl'` → mỗi kết quả là `(head, body, count, support%, confidence%, lift)`, trong đó **`head` là vế phải (1 item)** và **`body` là vế trái (tuple)**, tức luật là `body => head`.

[7]:
```
rules = fim.apriori(tracts, target='r', supp=MIN_SUP_PCT, conf=MIN_CONF_PCT,
                    zmin=2, report='aSCl')

rules_df = pd.DataFrame([
    {
        "rule": "{" + ", ".join(sorted(body)) + "} => {" + head + "}",
```

```
        "count": cnt,
        "support": sup / 100,
        "confidence": conf / 100,
        "lift": lift,
    }
    for head, body, cnt, sup, conf, lift in rules
]).sort_values(["confidence", "support", "lift"], ascending=False).
 ↪reset_index(drop=True)

print(f"Số luật (minSup {MIN_SUP_PCT}%, minConf {MIN_CONF_PCT}%):",␣
 ↪len(rules_df))
rules_df
```

```
Số luật (minSup 40%, minConf 60%): 24
```

[7]:

	rule	count	support	confidence	lift
0	{Beer} => {Diaper}	3	0.6	1.000000	1.250000
1	{Coke, Milk} => {Diaper}	2	0.4	1.000000	1.250000
2	{Coke, Diaper} => {Milk}	2	0.4	1.000000	1.250000
3	{Coke} => {Milk}	2	0.4	1.000000	1.250000
4	{Coke} => {Diaper}	2	0.4	1.000000	1.250000
5	{Beer, Bread} => {Diaper}	2	0.4	1.000000	1.250000
6	{Beer, Milk} => {Diaper}	2	0.4	1.000000	1.250000
7	{Diaper} => {Beer}	3	0.6	0.750000	1.250000
8	{Bread} => {Milk}	3	0.6	0.750000	0.937500
9	{Milk} => {Bread}	3	0.6	0.750000	0.937500
10	{Bread} => {Diaper}	3	0.6	0.750000	0.937500
11	{Diaper} => {Bread}	3	0.6	0.750000	0.937500
12	{Milk} => {Diaper}	3	0.6	0.750000	0.937500
13	{Diaper} => {Milk}	3	0.6	0.750000	0.937500
14	{Diaper, Milk} => {Coke}	2	0.4	0.666667	1.666667
15	{Bread, Diaper} => {Beer}	2	0.4	0.666667	1.111111
16	{Diaper, Milk} => {Beer}	2	0.4	0.666667	1.111111
17	{Beer, Diaper} => {Bread}	2	0.4	0.666667	0.833333
18	{Beer} => {Bread}	2	0.4	0.666667	0.833333
19	{Beer, Diaper} => {Milk}	2	0.4	0.666667	0.833333
20	{Beer} => {Milk}	2	0.4	0.666667	0.833333
21	{Bread, Milk} => {Diaper}	2	0.4	0.666667	0.833333
22	{Bread, Diaper} => {Milk}	2	0.4	0.666667	0.833333
23	{Diaper, Milk} => {Bread}	2	0.4	0.666667	0.833333

[8]:
```
print(f"{'LUẬT':<45}{'SUPPORT':>9}{'CONFIDENCE':>13}{'LIFT':>8}")
print("-" * 75)
for _, r in rules_df.iterrows():
    print(f"{r['rule']:<45}{r['support']:>9.2f}{r['confidence']:>13.
 ↪2f}{r['lift']:>8.2f}")
```

```
LUẬT                                          SUPPORT    CONFIDENCE    LIFT
```

```
------------------------------------------------------------------------
{Beer} => {Diaper}                          0.60         1.00    1.25
{Coke, Milk} => {Diaper}                    0.40         1.00    1.25
{Coke, Diaper} => {Milk}                    0.40         1.00    1.25
{Coke} => {Milk}                            0.40         1.00    1.25
{Coke} => {Diaper}                          0.40         1.00    1.25
{Beer, Bread} => {Diaper}                   0.40         1.00    1.25
{Beer, Milk} => {Diaper}                    0.40         1.00    1.25
{Diaper} => {Beer}                          0.60         0.75    1.25
{Bread} => {Milk}                           0.60         0.75    0.94
{Milk} => {Bread}                           0.60         0.75    0.94
{Bread} => {Diaper}                         0.60         0.75    0.94
{Diaper} => {Bread}                         0.60         0.75    0.94
{Milk} => {Diaper}                          0.60         0.75    0.94
{Diaper} => {Milk}                          0.60         0.75    0.94
{Diaper, Milk} => {Coke}                    0.40         0.67    1.67
{Bread, Diaper} => {Beer}                   0.40         0.67    1.11
{Diaper, Milk} => {Beer}                    0.40         0.67    1.11
{Beer, Diaper} => {Bread}                   0.40         0.67    0.83
{Beer} => {Bread}                           0.40         0.67    0.83
{Beer, Diaper} => {Milk}                    0.40         0.67    0.83
{Beer} => {Milk}                            0.40         0.67    0.83
{Bread, Milk} => {Diaper}                   0.40         0.67    0.83
{Bread, Diaper} => {Milk}                   0.40         0.67    0.83
{Diaper, Milk} => {Bread}                   0.40         0.67    0.83
```

[9]:
```
# Các luật mạnh nhất: confidence = 100% và lift > 1
rules_df[(rules_df["confidence"] >= 1.0) & (rules_df["lift"] > 1)].
↪reset_index(drop=True)
```

[9]:

	rule	count	support	confidence	lift
0	{Beer} => {Diaper}	3	0.6	1.0	1.25
1	{Coke, Milk} => {Diaper}	2	0.4	1.0	1.25
2	{Coke, Diaper} => {Milk}	2	0.4	1.0	1.25
3	{Coke} => {Milk}	2	0.4	1.0	1.25
4	{Coke} => {Diaper}	2	0.4	1.0	1.25
5	{Beer, Bread} => {Diaper}	2	0.4	1.0	1.25
6	{Beer, Milk} => {Diaper}	2	0.4	1.0	1.25

1.5 6. Đối chiếu với bản Apriori tự cài đặt ở bài 6.2

Cài lại gọn phần tự viết để so sánh kết quả.

[10]:
```
from itertools import combinations

D = [set(t) for t in tracts]

def my_apriori(D, min_support):
```

```
    n = len(D)
    items = sorted({i for t in D for i in t})
    freq, Ck, k = {}, [frozenset([i]) for i in items], 1
    while Ck:
        Fk = {c: sum(1 for t in D if c <= t) for c in Ck}
        Fk = {c: v for c, v in Fk.items() if v / n >= min_support}
        if not Fk:
            break
        freq.update({c: v / n for c, v in Fk.items()})
        prev = sorted(tuple(sorted(s)) for s in Fk)
        Ck = [frozenset(a) | frozenset(b)
              for i, a in enumerate(prev) for b in prev[i+1:]
              if a[:k-1] == b[:k-1] and len(frozenset(a) | frozenset(b)) == k +␣
 ↪1]
        Ck = [c for c in Ck if all(frozenset(s) in Fk for s in␣
 ↪combinations(sorted(c), k))]
        k += 1
    return freq

my_freq = my_apriori(D, MIN_SUP_PCT / 100)
fim_freq = {frozenset(s): pct / 100 for s, cnt, pct in itemsets}

print(f"Tự cài đặt : {len(my_freq)} tập phổ biến")
print(f"PyFIM      : {len(fim_freq)} tập phổ biến")
print("Trùng khớp itemsets      :", set(my_freq) == set(fim_freq))
print("Trùng khớp giá trị support:", all(abs(my_freq[s] - fim_freq[s]) < 1e-9␣
 ↪for s in my_freq))
```

```
Tự cài đặt : 17 tập phổ biến
PyFIM      : 17 tập phổ biến
Trùng khớp itemsets      : True
Trùng khớp giá trị support: True
```

[11]:
```
# Luật: PyFIM mặc định chỉ sinh luật có VẾ PHẢI đúng 1 item
def my_rules(freq, min_conf, head_size=None):
    out = []
    for I, s in freq.items():
        if len(I) < 2:
            continue
        for r in range(1, len(I)):
            if head_size and len(I) - r != head_size:
                continue
            for ante in combinations(sorted(I), r):
                X = frozenset(ante); Y = I - X
                if s / freq[X] >= min_conf:
                    out.append((X, Y))
    return out
```

```
all_rules    = my_rules(my_freq, MIN_CONF_PCT / 100)
single_head  = my_rules(my_freq, MIN_CONF_PCT / 100, head_size=1)
fim_pairs    = {(frozenset(body), frozenset([head])) for head, body, *_ in␣
 ↪rules}

print(f"Tự cài đặt (vế phải tuỳ ý)   : {len(all_rules)} luật")
print(f"Tự cài đặt (vế phải 1 item)  : {len(single_head)} luật")
print(f"PyFIM                        : {len(rules)} luật")
print("\nTrùng khớp khi giới hạn vế phải 1 item:", set(single_head) ==␣
 ↪fim_pairs)
```

```
Tự cài đặt (vế phải tuỳ ý)   : 27 luật
Tự cài đặt (vế phải 1 item)  : 24 luật
PyFIM                        : 24 luật

Trùng khớp khi giới hạn vế phải 1 item: True
```